Question 3: Docker Storage - Demonstrate loss of container-layer data after container deletion, and permanent storage using a Docker named volume and bind mount.

Three storage demos were performed: (1) data written inside a container is lost when the container is deleted, (2) a named volume survives container deletion, (3) a bind mount syncs a host directory with the container.

Demo 1 - container-layer data is lost

```
$ docker exec demo-layer sh -c "echo important-data > /tmp/note.txt && cat /tmp/note.txt"
important-data

$ docker rm -f demo-layer && docker run -d --name demo-layer ubuntu:24.04 sleep infinity
$ docker exec demo-layer cat /tmp/note.txt
cat: /tmp/note.txt: No such file or directory
--> file is GONE: container-layer data does not survive container deletion
```

Demo 2 - named volume survives

```
$ docker volume create lab2data
$ docker run -d --name demo-vol -v lab2data:/data ubuntu:24.04 sleep infinity
$ docker exec demo-vol sh -c "echo persistent-data > /data/note.txt"
$ docker rm -f demo-vol && docker run -d --name demo-vol -v lab2data:/data ubuntu:24.04 sleep infinity
$ docker exec demo-vol cat /data/note.txt
persistent-data
--> file SURVIVED: the volume lives in /var/lib/docker/volumes/lab2data/_data
```

Demo 3 - bind mount

```
$ echo "edited from the host" > ~/lab2-bindmount-data/host-note.txt
$ docker run --rm -v ~/lab2-bindmount-data:/data ubuntu:24.04 cat /data/host-note.txt
edited from the host

$ docker run --rm -v ~/lab2-bindmount-data:/data ubuntu:24.04 sh -c \
    "echo written-by-container >> /data/host-note.txt"
$ cat ~/lab2-bindmount-data/host-note.txt   (on the host)
edited from the host
written-by-container
--> both sides see the same file
```

```
abhinav@LAPTOP-BSV7PCJK: docs/commands  —  terminal
### Demo 1: data in the container layer is LOST when the container is deleted
abhinav@LAPTOP-BSV7PCJK:~$ docker rm -f demo-layer
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name demo-layer ubuntu:24.04 sleep infinity
Unable to find image 'ubuntu:24.04' locally
24.04: Pulling from library/ubuntu
7a48b8f0e5c7: Pulling fs layer
98144013299c: Download complete
7a48b8f0e5c7: Download complete
7a48b8f0e5c7: Pull complete
Digest: sha256:fe81560990fae86dc4743202b2cb4d3df98d0bcf949173cecef15ce1127ef852
Status: Downloaded newer image for ubuntu:24.04
c8cbc28ba7efe20c2c184917a0011990b3071f1fece42c919976d17faedce2ee
abhinav@LAPTOP-BSV7PCJK:~$ docker exec demo-layer sh -c "echo important-data > /tmp/note.txt && cat /tmp/n
important-data
abhinav@LAPTOP-BSV7PCJK:~$ docker rm -f demo-layer   # container deleted
demo-layer
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name demo-layer ubuntu:24.04 sleep infinity && docker exec demo
f82d9ad0777a2d9524cf6b50d025ef71565a4ce3a96f2ddcc09ced17e657d6ce
cat: /tmp/note.txt: No such file or directory
--> file is GONE: container-layer data does not survive container deletion
```

Fig: Demo 1: container-layer data lost after deletion

```
abhinav@LAPTOP-BSV7PCJK: docs/commands  —  terminal
### Demo 2: a named volume SURVIVES container deletion
abhinav@LAPTOP-BSV7PCJK:~$ docker volume create lab2data
lab2data
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name demo-vol -v lab2data:/data ubuntu:24.04 sleep infinity
c5e2183b5e8a4933f73199f9ef56a68767b0a923e3447e85ad75b03d3a07a625
abhinav@LAPTOP-BSV7PCJK:~$ docker exec demo-vol sh -c "echo persistent-data > /data/note.txt && cat /data/
persistent-data
abhinav@LAPTOP-BSV7PCJK:~$ docker rm -f demo-vol && docker run -d --name demo-vol -v lab2data:/data ubuntu
demo-vol
df34cc335ea2592ec4cffaf44142fdd4672dbcf9df3a0e9758cfe14633f3be93
abhinav@LAPTOP-BSV7PCJK:~$ docker exec demo-vol cat /data/note.txt
persistent-data
--> file SURVIVED: named volume lives outside the container lifecycle
abhinav@LAPTOP-BSV7PCJK:~$ docker volume inspect lab2data | grep -A2 Mountpoint
        "Mountpoint": "/var/lib/docker/volumes/lab2data/_data",
        "Name": "lab2data",
        "Options": null,
```

Fig: Demo 2: named volume survives container deletion

```
abhinav@LAPTOP-BSV7PCJK: docs/commands  —  terminal
### Demo 3: a bind mount maps a host directory into the container
abhinav@LAPTOP-BSV7PCJK:~$ echo "edited from the host" > ~/lab2-bindmount-data/host-note.txt
abhinav@LAPTOP-BSV7PCJK:~$ docker run --rm -v ~/lab2-bindmount-data:/data ubuntu:24.04 cat /data/host-note
edited from the host
abhinav@LAPTOP-BSV7PCJK:~$ docker run --rm -v ~/lab2-bindmount-data:/data ubuntu:24.04 sh -c "echo written
abhinav@LAPTOP-BSV7PCJK:~$ cat ~/lab2-bindmount-data/host-note.txt   (on the host)
edited from the host
written-by-container
--> both sides see the same file: bind mount is synced with the host filesystem
```

Fig: Demo 3: bind mount two-way sync

Observations

- Writable container layers are ephemeral by design; anything that must survive belongs in a volume.
- Named volumes are managed by Docker (under /var/lib/docker/volumes/); bind mounts are plain host paths.
- Bind mounts are ideal for development (edit on host, see changes in container).

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Docker, Docker Compose, Minikube and kubectl. The evidence (command logs, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-2 under the docs/ directory.